

Deploying PyTorch ML Workloads with Docker & Kubernetes

MLOps & Infrastructure for Machine Learning

Chintapalli Dharmendra
Roll No. DA25M559

August 30, 2026

Overview

This report accompanies the `mlops-pytorch-pipeline` submission for the individual assignment “*Deploying PyTorch ML Workloads with Docker & Kubernetes*”. The assignment’s goal was to take a PyTorch image classifier through a full deployment lifecycle: a proper Git workflow, a training/serving implementation in PyTorch, multi-stage Docker images for both, and orchestration of the same workload on Kubernetes using a `Job` for training and a `Deployment/Service` for serving.

Repository: <https://github.com/Chintapallidharmendra/mlops-pytorch-pipeline>

What Was Built

- Repository & Git workflow (Part A)

- Standard project layout (`src/`, `configs/`, `docker/`, `k8s/`, `requirements/`, `tests/`)
- a `develop` branch cut from `main`
- feature branches merged through pull requests with descriptions, following Conventional Commits.

Pull Request (Project Folder setup):

<https://github.com/Chintapallidharmendra/mlops-pytorch-pipeline/pull/1>

- PyTorch model (Part B)

- `src/model.py`: fine-tunes a `torchvision` ResNet-18 for CIFAR-10 (with a `simple_cnn` fallback architecture)
- `src/dataset.py`: handles `torchvision.datasets.CIFAR10` loading, augmentation, and `DataLoader` setup
- `src/train.py`: reads `configs/training_config.yaml`, logs JSON-lines metrics per epoch, saves the best checkpoint by validation loss, and supports early stopping
- `src/serve.py`: is a FastAPI app exposing `GET /health` and `POST /predict`

Pull Request (Model training and serve):

<https://github.com/Chintapallidharmendra/mlops-pytorch-pipeline/pull/2>

- **Docker containerization (Part C)** `docker/Dockerfile.train` and `docker/Dockerfile.serve` are both two-stage builds with pinned dependencies (`requirements/train.txt`, `requirements/serve.txt`); the serving image installs inference-only dependencies, exposes port 8080, runs as a non-root user, and includes a `HEALTHCHECK`.

Pull Request (Docker containerization):

<https://github.com/Chintapallidharmendra/mlops-pytorch-pipeline/pull/3>

- **Kubernetes training Job (Part D)** `k8s/namespace.yaml`, `k8s/configmap.yaml`, and `k8s/training-job.yaml` run training as a Job in the `ml-training` namespace, mounting the config as a volume and using `PersistentVolumeClaims` for data and checkpoints, with CPU/memory requests and limits set as specified.
- **Kubernetes serving (Part E)** `k8s/serving-deployment.yaml` runs two replicas behind a rolling-update strategy, mounts the checkpoint PVC read-only, and configures liveness/readiness probes against `/health`; `k8s/serving-service.yaml` exposes it as a `ClusterIP` service.
- **End-to-end validation (Part F)** The full workflow with apply manifests, wait for the training Job, deploy serving, port-forward, and call `/predict` was run against a local cluster; terminal output and screenshots are captured under `proofs/`.

Pull Request (Kubernetes all changes with proofs attached):

<https://github.com/Chintapallidharmendra/mlops-pytorch-pipeline/pull/5>

Architecture

Figure 1 traces one artifact (the trained checkpoint) from source code, through the Docker images, into the Kubernetes cluster, and out to a client request.

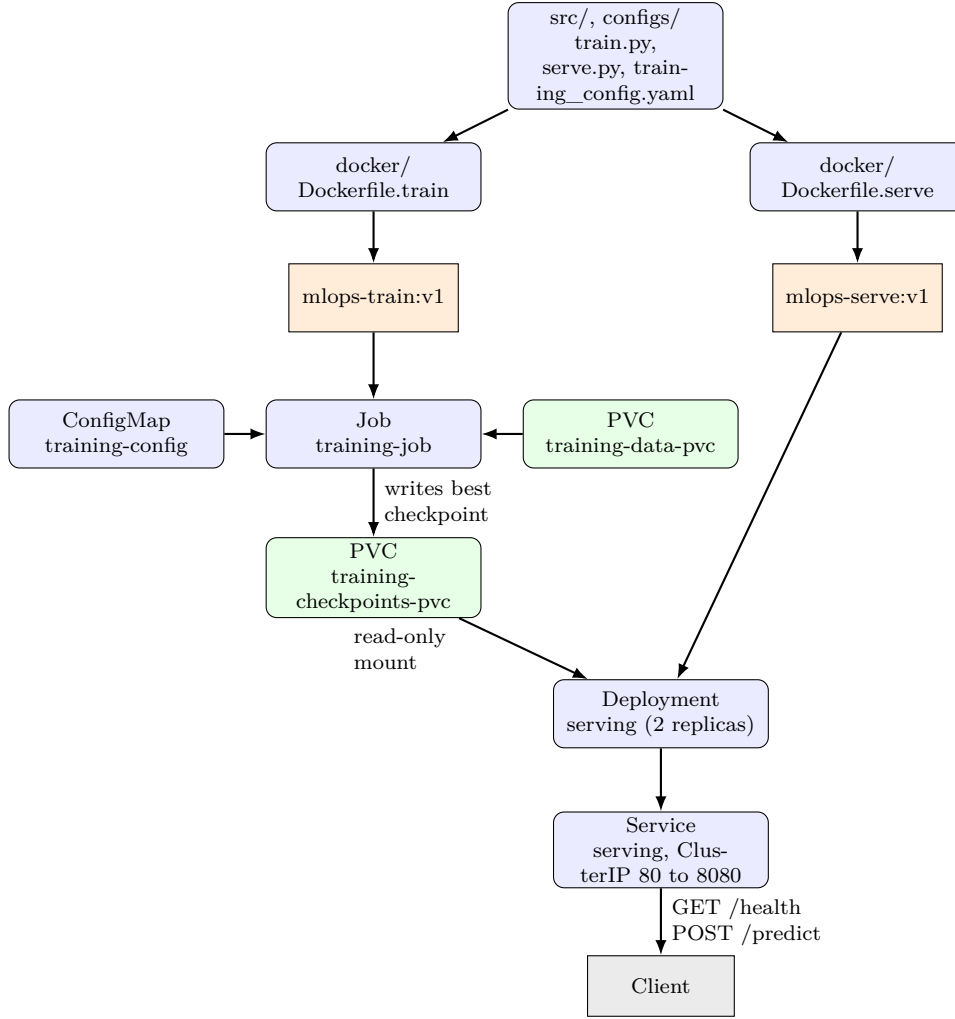


Figure 1: Handoff of the trained checkpoint from source, through Docker images, into the Kubernetes namespace `ml-training`, and out to a client.

Reflection: What Was the Most Challenging Part

The most challenging part of this assignment was not training the model itself (a ResNet-18 fine-tuned on CIFAR-10 converges without much drama) but making the same source code behave identically across four different execution contexts: a bare Python interpreter, a Docker container, a Kubernetes Job, and a Kubernetes Deployment.

The first surprise showed up at the Docker layer. `train.py` and `serve.py` both use bare imports (`from dataset import get_dataloaders`, `from model import get_model`) rather than package-relative ones, which only resolve because Python adds the directory of the launched script to `sys.path`. That meant the Dockerfile’s `WORKDIR`, the `COPY` destinations, and the exact form of the `ENTRYPOINT` all had to line up precisely running `python src/train.py` from `/app` works, but `cd src && python train.py`, or launching `uvicorn` without `-app-dir src`, silently breaks the sibling imports. A related, less obvious failure was PyTorch’s Linux wheel dynamically linking against `libgomp.so.1` at import time, a library `python:3.11-slim-bookworm` does not ship. The image would build cleanly and only fail at runtime with an opaque `OSError`, so `libgomp1` had to be installed explicitly in the base stage before `pip install` was even worth running. Getting the CNN backbone right needed similar care: a stock `torchvision` ResNet-18 is built for 224×224 ImageNet input, so its 7×7 stride-2 stem plus max-pool collapse a 32×32 CIFAR-

10 image almost to nothing before the first residual block. `conv1` and the max-pool had to be replaced for the smaller input, which also meant accepting that only blocks two onward keep their pretrained ImageNet weights.

The hardest part, though, was the Kubernetes hand-off between training and serving. Locally, a checkpoint is just a file in `checkpoints/`; in the cluster, the training `Job` and the serving `Deployment` are separate pods, possibly on separate nodes, that only agree on a shared `PersistentVolumeClaim`. That forced design decisions the local setup never demanded: the `Job` must run to completion and populate `training-checkpoints-pvc` before the `Deployment` is applied, the `Deployment` mounts that same PVC read-only so replicas cannot corrupt it, and the readiness/liveness probes need a large enough `initialDelaySeconds` that a pod is not killed while the checkpoint is still loading into memory. None of this is visible in `serve.py` itself. It only shows up in how the manifests are sequenced and applied, which made it easy to write YAML that looked correct but failed the moment `kubectl apply` reached Job/Deployment ordering.

Overall, the exercise reinforced that most of the real “MLOps” difficulty is not the model but it is making the boundaries between environments (local, container, cluster) invisible to the code that runs inside them.

Conclusion

The final submission satisfies Parts A through F of the assignment: a Git-disciplined repository, a CIFAR-10 classifier with training and FastAPI serving, pinned multi-stage Docker images, and Kubernetes manifests that carry a checkpoint from an ephemeral training `Job` to a replicated, health-checked serving `Deployment`.